

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и компьютерной техники

Лабораторная работа №4
по дисциплине «Основы программной инженерии»
Вариант: 58532

Преподаватель:
Лазеев Сергей Максимович

Выполнил:
Караганов Павел Эдуардович
Группа: Р3210

Содержание

1 Текст задания	3
2 Сбор метрик с помощью Prometheus и Grafana	3
2.1 Конфигурация целей в Prometheus	3
2.2 Результаты нагрузочного тестирования в Yandex Tank	3
2.3 Интеграционный дашборд Grafana	4
3 Результаты профилирования в IntelliJ IDEA	4
4 Анализ использования памяти и поиск утечек	5
5 Вывод по работе	6

1 Текст задания

Цель работы: Изучение механизмов мониторинга, профилирования и нагрузочного тестирования корпоративных Java-приложений.

Задание: Для разработанного веб-приложения (Лабораторная работа №4) реализовать:

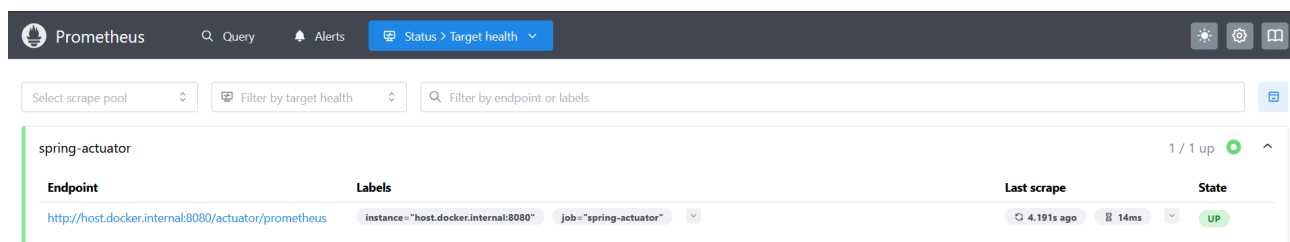
1. Интеграция современной системы сбора метрик Prometheus и платформы визуализации Grafana с использованием Docker-контейнеров.
2. Проведение нагрузочного тестирования с помощью утилиты Yandex Tank с имитацией многопользовательского доступа по JWT-токенам.
3. Локализация узких мест производительности с помощью встроенного в IntelliJ IDEA профилировщика.

2 Сбор метрик с помощью Prometheus и Grafana

Для мониторинга была развернута связка систем из контейнеров Docker. Сбор метрик со стороны Spring Boot осуществлялся с помощью модуля spring-boot-starter-actuator и библиотеки micrometer-registry-prometheus.

2.1 Конфигурация целей в Prometheus

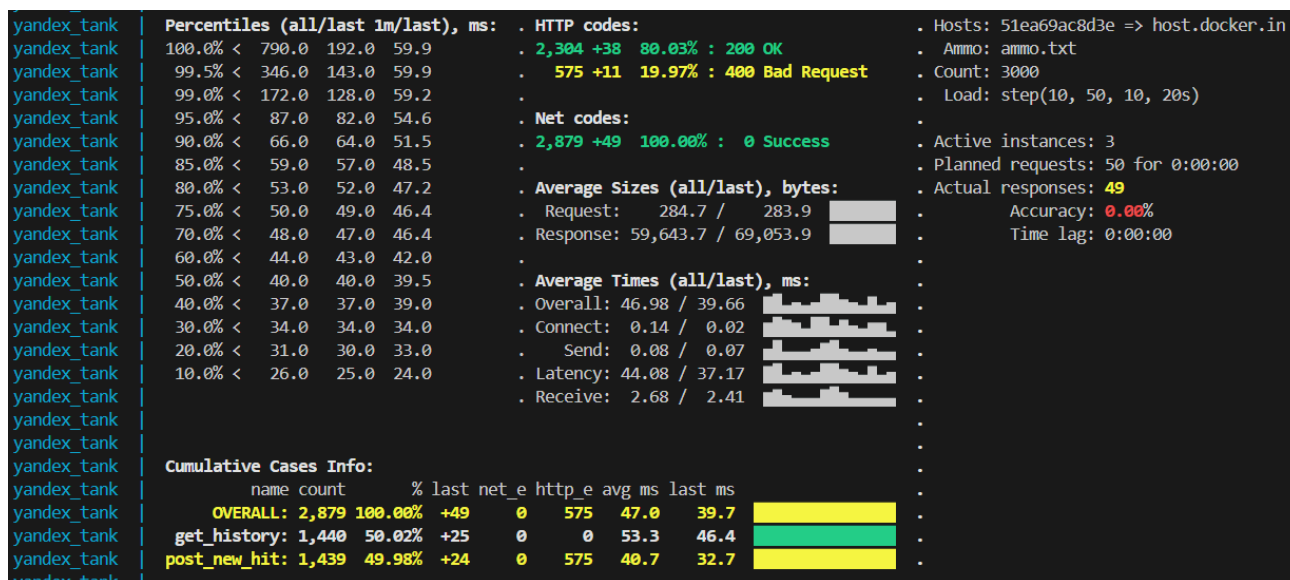
Сборщик метрик Prometheus настроен на регулярный опрос (скрейпинг) эндпоинта приложения с интервалом в 5 секунд.



На рисунке 1 видно, что цель spring-actuator успешно определена и имеет статус UP. Сбор системных и прикладных метрик проходит без сетевых ошибок.

2.2 Результаты нагрузочного тестирования в Yandex Tank

Нагрузочное тестирование проводилось с помощью утилиты Yandex Tank. Был подготовлен сценарий ступенчатой нагрузки, генерирующий до 50 запросов в секунду (RPS) от имени нескольких тестовых пользователей с динамической генерацией JWT-токенов в заголовке Authorization.

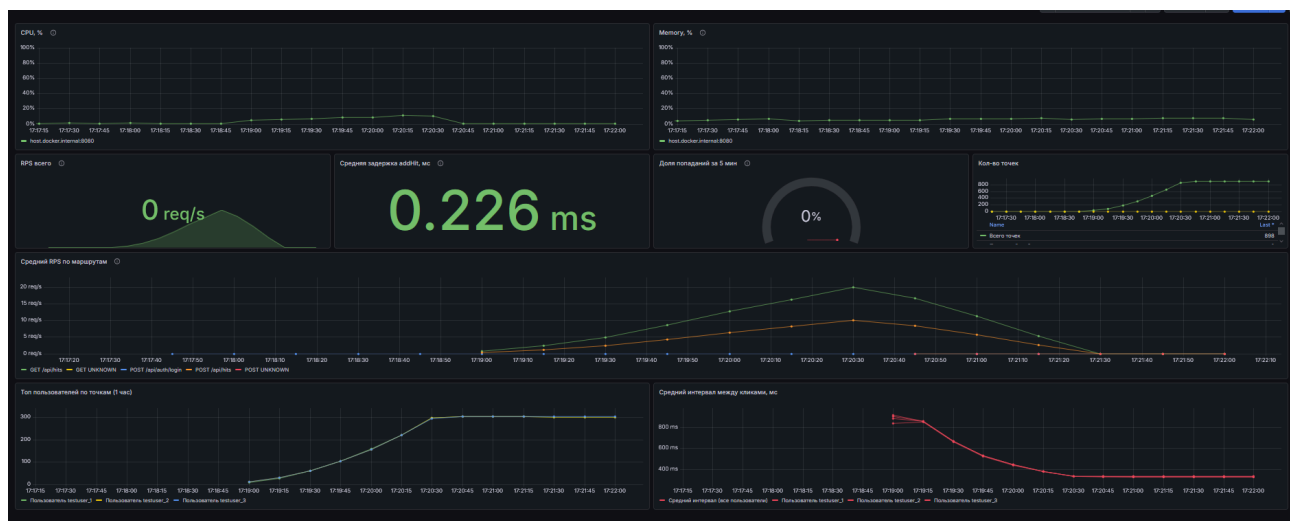


Анализ результатов обстрела (Рисунок 2):

- Сетевой уровень (Net codes) показал 100% стабильность (0 ошибок).
- Около 80% запросов вернули статус 200 OK. 20% запросов вернули статус 400 Bad Request. Это обусловлено работой DTO-валидаторов на бэкенде, которые отсекали невалидные координаты, сгенерированные скриптом подготовки патронов.
- Медианное время ответа (50% percentile) составило всего 31 мс, что подтверждает высокую отзывчивость системы.

2.3 Интеграционный дашборд Grafana

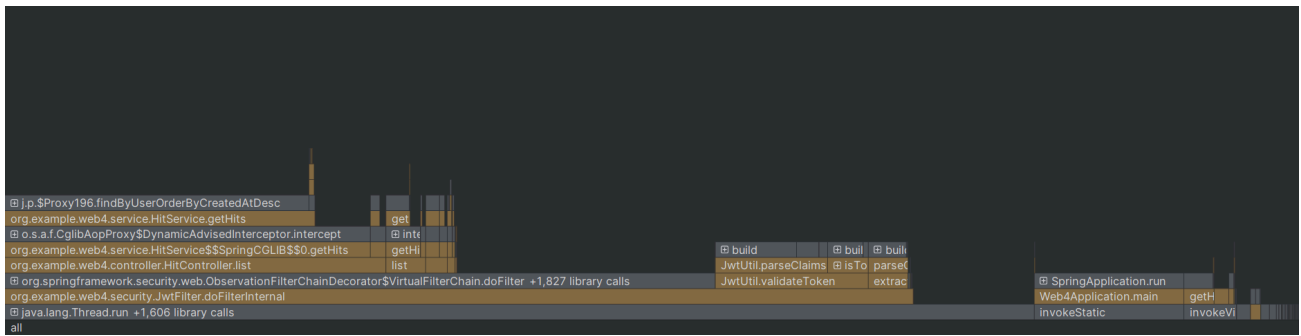
Для визуализации метрик под нагрузкой был разработан кастомный дашборд в Grafana.



На дашборде (Рисунок 3) наглядно виден ступенчатый рост графиков RPS и коррелирующий с ним рост прикладных метрик количества точек.

3 Результаты профилирования в IntelliJ IDEA

Локализация узких мест производительности проводилась с помощью встроенного в среду разработки IntelliJ IDEA профилировщика на базе ядра async-profiler во время активного обстрела приложения Танком.

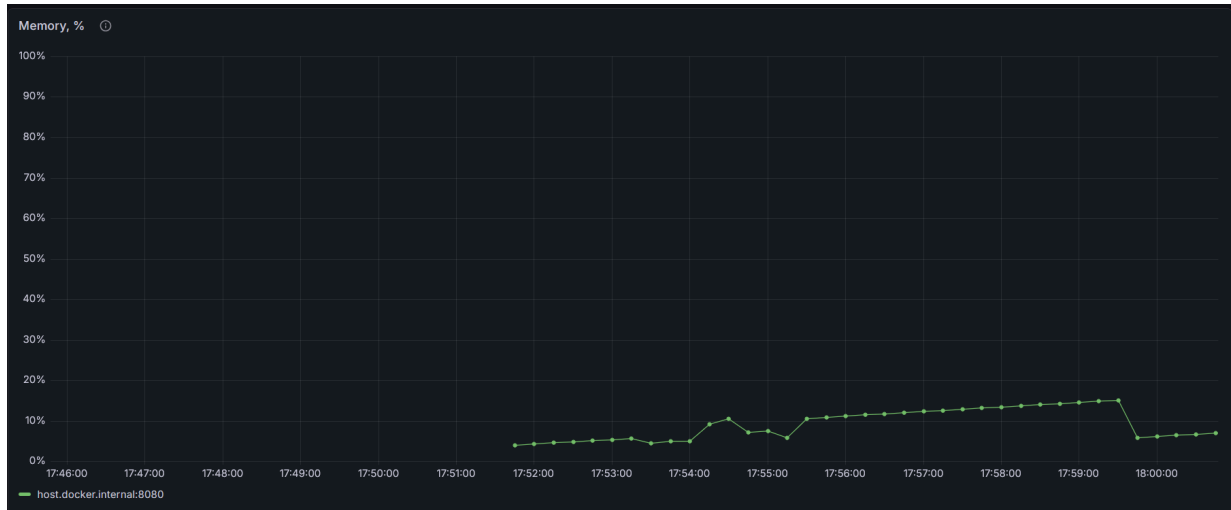


Анализ пламенного графика (Рисунок 4):

- По оси Y отложена глубина стека вызовов, по оси X — относительное процессорное время, затраченное на метод.
- Узкое место №1 (Ввод-вывод): Широкий блок метода `findByUserOrderByCreatedAtDesc` в левой части показывает, что приложение тратит значительное время на ожидания синхронных ответов от базы данных PostgreSQL. Возможный способ решения: индексация полей `user_id` и `created_at` в БД.
- Узкое место №2 (Криптография): Широкий блок `JwtUtil.validateToken` в центре показывает высокую нагрузку на CPU от операций валидации подписей JWT. Возможный способ решения: кэширование результатов валидации токенов в оперативной памяти (например, Redis).

4 Анализ использования памяти и поиск утечек

Для подтверждения стабильности работы приложения и отсутствия утечек памяти (Memory Leaks) был проанализирован график потребления кучи (JVM Heap Memory).



Выводы по анализу памяти (Рисунок 5): График использования памяти имеет классический здоровый пилообразный вид.

- Нарастание зубьев пилы отражает создание временных объектов (DTO, HTTP-запросы) во время обстрела Танком.
- Резкие спады вниз отражают моменты успешного срабатывания GC, который полностью высвобождает неиспользуемую память. Отсутствие монотонного роста нижней границы «пилы» доказывает, что в приложении нет утечек памяти.

5 Вывод по работе

В ходе выполнения лабораторной работы были освоены фундаментальные методы верификации и анализа производительности программных систем.

Была успешно спроектирована и реализована система мониторинга с помощью современного стека сбора и визуализации метрик на базе Prometheus и Grafana в контейнерах Docker. Использование утилиты Yandex Tank позволило провести полноценный стресс-тест бэкенда в условиях авторизованного многопоточного доступа с JWT-токенами.

Инструменты профилирования (встроенный профилировщик IntelliJ IDEA) позволили визуализировать стек вызовов в виде пламенного графика, локализовать ресурсоемкие участки кода (криптографическая проверка JWT и обращения к реляционной БД) и доказать отсутствие утечек памяти в JVM за счет анализа пилообразного характера графиков утилизации кучи.